

ggerganov /
llama.cpp[Code](#) [Issues 302](#) [Pull requests 236](#) [Discussions](#) [Actions](#) [Projects 8](#) [Wiki](#) [Security 1](#)

What does the parameter means and their impact of the output speed ? #559

Unanswered

Sh3yn3 asked this question in Q&A



Sh3yn3 on Mar 28, 2023

Hello there,

So there are plenty parameters, and for a lot of them i have no clue about what they are used for or if they can help me to have better answer time. I would like to have a realistic chatbot, that doesn't take 2-5 min to answer a single question, but still doesn't sound like a robot. It would be awesome to have a place where we could resume those things for the newcomer like myself, so maybe you could help me know more or to correct my mistakes :

temp : (This one doesn't seem to impact on the speed, since it just change how much the AI will stay on the topic)

top_k : (This is the number of probable next words, to create a pool of words to choose from)

top_p : (This is by how much a word should be probable to be picked)

repeat_last_n : (I have no idea)

repeat_penalty : (Seems like the higher the less the AI will repeat itself)

n_ctx : (I don't know what this exactly is and how much it helps to speed stuff)

n_batch : (That's the amount of character the AI can compute in the same round, it seems like you should keep it as low as possible for your needs)

n_predict : (I believe that's how many character the AI think upfront before talking, if it's lower than the sentence it may have to compute another round or talk nonsense)

n_keep : (I have no idea)

Could someone please point me toward what repeat_last_n, n_keep and n_ctx do ? And tell me what parameter really play in the speed of the answer ?

So far i'm using that without too much idea of what i'm doing with extremely low performance :

```
sampling: temp = 0.700000, top_k = 40, top_p = 0.500000, repeat_last_n = 256, repeat_penalty = 1.176470  
generate: n_ctx = 2048, n_batch = 512, n_predict = 2048, n_keep = 0
```



Thank you !



2 comments · 1 reply

Oldest

Newest | Top



j-f1 on Mar 28, 2023

Collaborator

The only things that would affect inference speed are model size (7B is fastest, 65B is slowest) and your CPU/RAM specs.

n_ctx sets the maximum length of the prompt and output combined (in tokens), and n_predict sets the maximum number of tokens the model will output after outputting the prompt.

repeat_last_n controls how large the window of tokens is that the model will be penalized for repeating (repeat_penalty sets the amount the model will be penalized for attempting to use one of those tokens).

n_batch only affects the phase where the model is ingesting the prompt. You might find that part runs faster if you increase it. It has no effect on the part where the model generates new output.

n_keep is used when the n_ctx limit is reached. A new prompt will be constructed with the first n_keep characters of the original prompt plus the second half of the output to free up space for more conversation.



0 replies



luminalle on Mar 29, 2023

edited ▾

Here is my inaccurate list I constructed from iffy sources (mostly this thread). I hope someone can point the mistakes or suggest better explanations.

--temp - controls looseness on prompt or how wild/creative the AI is
--top_k - number of most likely next words in a pool to choose from
--top_p - how probable the word has to be to get picked?
--ctx_size - maximum length of the prompt and output combined (in tokens)
--n_predict - maximum number of tokens the model will output after outputting the prompt - number of tokens to predict
--keep - number of tokens to keep from the initial prompt - when the n_ctx limit is reached
--repeat_last_n - last n tokens to consider for penalize - size of window of tokens that the model will be penalized for repeating
--repeat_penalty - sets the amount the model will be penalized for attempting to use one of those tokens



1 reply



j-f1 on Mar 29, 2023

Collaborator

That looks good! I think the underlying behavior of temperature is that it controls how often the model will choose a token that is not the most probable next token (0 means it always uses the most probable option, which turns out not to be the best idea all the time).



Category



Q&A

Labels

None yet

3 participants

